

```

import { useState, useEffect, useRef } from "react";

const CLAUDE_MODEL = "claude-sonnet-4-20250514";

// ——— ICONS ———
const Icon = ({ d, size = 20, stroke = 2 }) => (
  <svg width={size} height={size} viewBox="0 0 24 24" fill="none"
    stroke="currentColor" strokeWidth={stroke} strokeLinecap="round" strokeLinejoin="round">
    <path d={d} />
  </svg>
);

const Icons = {
  hard_hat: "M2 20h20M6 20v-6a6 6 0 0 1 12 0v6M3 14h18",
  plan: "M14 2H6a2 2 0 0 0-2 2v16a2 2 0 0 0 2 2h12a2 2 0 0 0 2-2V8zM14 2v6h6",
  workers: "M17 21v-2a4 4 0 0 0-4-4H5a4 4 0 0 0-4 4v2M9 11a4 4 0 1 0 0-8 4 4 0 1 0 0-8",
  materials: "M21 16V8a2 2 0 0 0-1-1.73l-7-4a2 2 0 0 0-2 0l-7 4A2 2 0 0 0 3 8v8a2 2 0 0 0 2 2",
  calendar: "M3 4h18v16H3zM16 2v4M8 2v4M3 10h18",
  alert: "M10.29 3.86L1.82 18a2 2 0 0 0 1.71 3h16.94a2 2 0 0 0 1.71-3L13.71 3.86",
  check: "M20 6L9 17l-5-5",
  send: "M22 2L11 13M22 21-7 20-4-9-9-4 20-7z",
  bot: "M12 2a2 2 0 0 1 2 2v1h3a1 1 0 0 1 1 1v3a1 1 0 0 1-1 1h-1v8a2 2 0 0 1 2 2",
  upload: "M21 15v4a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2v-4M17 8l-5-5-5 5M12 3v12",
  euro: "M14 5a7 7 0 1 0 0 14M4 9h10M4 15h10",
  clock: "M12 22a10 10 0 1 0 0-20 10 10 0 0 0 0 20zM12 6v6l4 2",
  chevron: "M9 18l6-6-6-6",
  logo: "M2 20h20M6 20V10l6-8 6 8v10M10 20v-5h4v5",
};

// ——— SYSTEM PROMPT ———
const buildSystemPrompt = (project) => `Tu es Construct AI, un agent IA expert en
  Tu assistes le chef de projet sur le chantier "${project.name}" avec les données
  - Budget total : ${project.budget}€
  - Délai : du ${project.startDate} au ${project.endDate}
  - Contraintes : ${project.constraints}
  - Description : ${project.description}

  Tes capacités :
  1. Commande d'ouvriers (plombiers, électriciens, maçons, etc.)
  2. Commande de matériaux (béton, acier, bois, etc.)
  3. Gestion du planning journalier
  4. Coordination des équipes et entreprises
  5. Vérification et contrôle des ouvrages
  6. Alertes et prévention des risques

```

Réponds toujours en français, de façon professionnelle mais directe.
Quand tu proposes une action concrète (commande, planning, etc.), formate-la clair
Sois proactif : anticipe les problèmes et propose des solutions.`;

```
// — ONBOARDING —  
function Onboarding({ onComplete }) {  
  const [step, setStep] = useState(0);  
  const [form, setForm] = useState({  
    name: "", description: "", budget: "", startDate: "", endDate: "", constraint  
  });  
  const [animating, setAnimating] = useState(false);  
  
  const fields = [  
    { key: "name", label: "Nom du chantier", placeholder: "Ex: Résiden  
    { key: "description", label: "Description du projet", placeholder: "Ex: Const  
    { key: "budget", label: "Budget total (€)", placeholder: "Ex: 2500000  
    { key: "startDate", label: "Date de début", placeholder: "",  
    { key: "endDate", label: "Date de fin prévue", placeholder: "",  
    { key: "constraints", label: "Contraintes", placeholder: "Ex: Bruit i  
  ];  
  
  const next = () => {  
    if (step < fields.length - 1) {  
      setAnimating(true);  
      setTimeout(() => { setStep(s => s + 1); setAnimating(false); }, 200);  
    } else {  
      onComplete(form);  
    }  
  };  
  
  const f = fields[step];  
  const progress = ((step + 1) / fields.length) * 100;  
  
  return (  
    <div style={styles.onboardingWrap}>  
      <div style={styles.onboardingCard}>  
        { /* Logo */ }  
        <div style={styles.onboardingLogo}>  
          <svg width="36" height="36" viewBox="0 0 24 24" fill="none" stroke="#F5  
            <path d={Icons.logo} />  
          </svg>  
          <span style={styles.onboardingLogoText}>Construct <span style={{color:"  
        </div>  
  
        { /* Progress */ }  
        <div style={styles.progressBar}>  
          <div style={{...styles.progressFill, width: `${progress}%`} />  
        </div>  
      </div>  
    </div>  
  );  
}
```

```

</div>
<p style={styles.progressLabel}>{step + 1} / {fields.length}</p>

{/* Field */}
<div style={{...styles.fieldWrap, opacity: animating ? 0 : 1, transform:
  <label style={styles.fieldLabel}>
    <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="#"
      <path d={f.icon} />
    </svg>
    {f.label}
  </label>
  {f.key === "description" || f.key === "constraints" ? (
    <textarea
      style={styles.textarea}
      placeholder={f.placeholder}
      value={form[f.key]}
      onChange={e => setForm(p => ({...p, [f.key]: e.target.value}))}
      rows={3}
    </>
  ) : (
    <input
      style={styles.input}
      type={f.type || "text"}
      placeholder={f.placeholder}
      value={form[f.key]}
      onChange={e => setForm(p => ({...p, [f.key]: e.target.value}))}
    </input>
  )}
</div>

<button style={styles.nextBtn} onClick={next}>
  {step < fields.length - 1 ? "Continuer →" : "🚀 Lancer Construct AI"}
</button>
</div>
</div>
);
}

// — CHAT —————
function Chat({ project }) {
  const [messages, setMessages] = useState([
    {
      role: "assistant",
      content: `Bonjour ! Je suis votre agent IA pour le chantier **"${project.name}"`
    }
  ]);
  const [input, setInput] = useState("");

```

```

const [loading, setLoading] = useState(false);
const endRef = useRef(null);

const SUGGESTIONS = [
  "Planifie la semaine prochaine",
  "Commande des matériaux pour les fondations",
  "Quels ouvriers faut-il pour demain ?",
  "Quel est l'état du budget ?",
  "Identifie les risques actuels",
];

useEffect(() => { endRef.current?.scrollIntoView({ behavior: "smooth" }); }, [m

const sendMessage = async (text) => {
  const userMsg = text || input.trim();
  if (!userMsg || loading) return;
  setInput("");
  const newMessages = [...messages, { role: "user", content: userMsg }];
  setMessages(newMessages);
  setLoading(true);

  try {
    const apiMessages = newMessages.map(m => ({ role: m.role, content: m.conten
    const response = await fetch("https://api.anthropic.com/v1/messages", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        model: CLAUDE_MODEL,
        max_tokens: 1000,
        system: buildSystemPrompt(project),
        messages: apiMessages,
      })),
    });
    const data = await response.json();
    const reply = data.content?.map(b => b.text || "").join("\n") || "Erreur de
    setMessages(prev => [...prev, { role: "assistant", content: reply }]);
  } catch {
    setMessages(prev => [...prev, { role: "assistant", content: "Erreur de conn
  } finally {
    setLoading(false);
  }
};

const renderContent = (text) => {
  return text.split("\n").map((line, i) => {
    if (line.startsWith("***") && line.endsWith("***")) {
      return <p key={i} style={{fontWeight:700, margin:"6px 0"}}>{line.slice(2,

```

```

    }
    if (line.startsWith("• ") || line.startsWith("- ")) {
      return <p key={i} style={{margin:"3px 0", paddingLeft:12}}>{"• " + line.s
    }
    // bold inline
    const parts = line.split(/\*\*(.*?)\*\*/g);
    return <p key={i} style={{margin:"3px 0"}}>{parts.map((p,j) => j%2===1 ? <s
  });
};

return (
  <div style={styles.chatWrap}>
    <div style={styles.messages}>
      {messages.map((m, i) => (
        <div key={i} style={{...styles.msgRow, justifyContent: m.role === "user"
          {m.role === "assistant" && (
            <div style={styles.botAvatar}>
              <svg width="16" height="16" viewBox="0 0 24 24" fill="none" strok
                <path d={Icons.bot} />
              </svg>
            </div>
          ))}
        <div style={m.role === "user" ? styles.userBubble : styles.botBubble}
          {renderContent(m.content)}
        </div>
      </div>
    )}}
    {loading && (
      <div style={{...styles.msgRow, justifyContent:"flex-start"}}>
        <div style={styles.botAvatar}>
          <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke=
            <path d={Icons.bot} />
          </svg>
        </div>
        <div style={styles.botBubble}><span style={styles.typing}>●●●</span><
      </div>
    )}
    <div ref={endRef} />
  </div>

  {/* Suggestions */}
  <div style={styles.suggestions}>
    {SUGGESTIONS.map((s, i) => (
      <button key={i} style={styles.suggBtn} onClick={() => sendMessage(s)}>{
    )}}
  </div>

```

```

    {/* Input */}
    <div style={styles.inputRow}>
      <input
        style={styles.chatInput}
        value={input}
        onChange={e => setInput(e.target.value)}
        onKeyDown={e => e.key === "Enter" && sendMessage()}
        placeholder="Demandez à Construct AI..."
      />
      <button style={{...styles.sendBtn, opacity: loading ? 0.5 : 1}} onClick={
        <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="whi
          <path d={Icons.send} />
        </svg>
      </button>
    </div>
  </div>
);
}

// ——— DASHBOARD ———
function Dashboard({ project, onOpenChat }) {
  const spent = Math.round(Number(project.budget) * 0.34);
  const pct = Math.round((spent / Number(project.budget)) * 100);

  const kpis = [
    { label: "Budget dépensé", value: ` ${(spent/1000).toFixed(0)}k€`, sub: ` / ${((
    { label: "Ouvriers actifs", value: "14", sub: "ce jour", color: "#4ADE80" },
    { label: "Tâches en cours", value: "7", sub: "sur 12 planifiées", color: "#60
    { label: "Alertes", value: "2", sub: "à traiter", color: "#F87171" },
  ];

  const tasks = [
    { name: "Coulage dalle R+1", status: "done", team: "Maçonnerie", date:
    { name: "Câblage électrique", status: "active", team: "Électricité", date:
    { name: "Plomberie - niveau 2", status: "active", team: "Plomberie", date:
    { name: "Livraison acier IPE", status: "pending", team: "Logistique", date:
    { name: "Inspection fondations", status: "done", team: "Contrôle", date:
  ];

  const statusStyle = {
    done: { bg: "#14532d", color: "#4ADE80", label: "✓ Terminé" },
    active: { bg: "#1e3a5f", color: "#60A5FA", label: "⚡ En cours" },
    pending: { bg: "#3b2a0e", color: "#F5A623", label: "🕒 Planifié" },
  };

  return (
    <div style={styles.dashWrap}>

```

```

    { /* Header */ }
    <div style={styles.dashHeader}>
      <div>
        <h2 style={styles.dashTitle}>{project.name}</h2>
        <p style={styles.dashSub}>{project.startDate} → {project.endDate} · Bud
      </div>
      <button style={styles.openChatBtn} onClick={onOpenChat}>
        <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="cur
          <path d={Icons.bot} />
        </svg>
        Parler à l'agent IA
      </button>
    </div>

    { /* KPIs */ }
    <div style={styles.kpiGrid}>
      {kpis.map((k, i) => (
        <div key={i} style={styles.kpiCard}>
          <p style={styles.kpiLabel}>{k.label}</p>
          <p style={{...styles.kpiValue, color: k.color}}>{k.value}</p>
          <p style={styles.kpiSub}>{k.sub}</p>
        </div>
      ))}
    </div>

    { /* Budget bar */ }
    <div style={styles.budgetCard}>
      <div style={{display:"flex", justifyContent:"space-between", marginBottom
        <span style={{color:"#ccc", fontSize:13}}>Avancement budgétaire</span>
        <span style={{color:"#F5A623", fontWeight:700}}>{pct}%</span>
      </div>
      <div style={styles.budgetTrack}>
        <div style={{...styles.budgetFill, width:`${pct}%`} />
      </div>
    </div>

    { /* Tasks */ }
    <div style={styles.sectionTitle}>Planning du jour</div>
    <div style={styles.taskList}>
      {tasks.map((t, i) => {
        const s = statusStyle[t.status];
        return (
          <div key={i} style={styles.taskRow}>
            <div style={{flex:1}}>
              <p style={styles.taskName}>{t.name}</p>
              <p style={styles.taskMeta}>{t.team} · {t.date}</p>
            </div>

```

```

        <span style={{...styles.badge, background: s.bg, color: s.color}}>{
      </div>
    );
  }}}
</div>

{/* Alerts */}
<div style={styles.sectionTitle}>Alertes actives</div>
[
  "Livraison béton retardée de 2 jours – impact planning à prévoir",
  "Effectif plombiers insuffisant jeudi – commander renforts",
].map((a, i) => (
  <div key={i} style={styles.alertRow}>
    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="#F8
      <path d={Icons.alert} />
    </svg>
    <p style={{margin:0, fontSize:13, color:"#fca5a5"}}>{a}</p>
  </div>
  )))
</div>
);
}

// ——— MAIN APP ———
export default function App() {
  const [project, setProject] = useState(null);
  const [view, setView] = useState("dashboard"); // dashboard | chat

  if (!project) return <Onboarding onComplete={setProject} />;

  return (
    <div style={styles.appWrap}>
      {/* Nav */}
      <nav style={styles.nav}>
        <div style={styles.navLogo}>
          <svg width="22" height="22" viewBox="0 0 24 24" fill="none" stroke="#F5
            <path d={Icons.logo} />
          </svg>
          <span>Construct <span style={{color:"#F5A623"}}>AI</span></span>
        </div>
        <div style={styles.navTabs}>
          [{"dashboard","chat"].map(v => (
            <button key={v} style={{...styles.navTab, ...(view===v ? styles.navTa
              {v === "dashboard" ? "🏗️ Chantier" : "🤖 Agent IA"}
            </button>
          )))
        </div>
      </nav>
    </div>
  );
}

```

```

    </nav>

    { /* Content */ }
    <main style={styles.main}>
      {view === "dashboard"
        ? <Dashboard project={project} onOpenChat={() => setView("chat")} />
        : <Chat project={project} />
      }
    </main>
  </div>
);
}

// ——— STYLES ———
const styles = {
  // Onboarding
  onboardingWrap: { minHeight:"100vh", background:"#0a0a0a", display:"flex", align
  onboardingCard: { background:"#141414", border:"1px solid #2a2a2a", borderRadiu
  onboardingLogo: { display:"flex", alignItems:"center", gap:10, marginBottom:32
  onboardingLogoText: { fontSize:22, fontWeight:700, color:"#fff", letterSpacing:
  progressBar: { height:3, background:"#1e1e1e", borderRadius:99, marginBottom:
  progressFill: { height:"100%", background:"linear-gradient(90deg,#F5A623,#f973
  progressLabel: { fontSize:12, color:"#555", marginBottom:28, textAlign:"right"
  fieldWrap: { marginBottom:24 },
  fieldLabel: { display:"flex", alignItems:"center", gap:8, color:"#aaa", font
  input: { width:"100%", background:"#1a1a1a", border:"1px solid #2a2a2a"
  textarea: { width:"100%", background:"#1a1a1a", border:"1px solid #2a2a2a"
  nextBtn: { width:"100%", background:"#F5A623", border:"none", borderRadiu

  // App shell
  appWrap: { minHeight:"100vh", background:"#0a0a0a", display:"flex", flexDirecti
  nav: { background:"#111", borderBottom:"1px solid #1e1e1e", padding:"12px 2
  navLogo: { display:"flex", alignItems:"center", gap:8, color:"#fff", fontSize:1
  navTabs: { display:"flex", gap:6 },
  navTab: { background:"transparent", border:"1px solid #2a2a2a", borderRadius:6
  navTabActive: { background:"#1a1a1a", border:"1px solid #F5A623", color:"#F5A62
  main: { flex:1, overflow:"auto" },

  // Dashboard
  dashWrap: { maxWidth:720, margin:"0 auto", padding:"24px 16px 40px" },
  dashHeader: { display:"flex", alignItems:"flex-start", justifyContent:"space-be
  dashTitle: { color:"#fff", fontSize:20, fontWeight:700, margin:"0 0 4px", lett
  dashSub: { color:"#666", fontSize:13, margin:0 },
  openChatBtn:{ background:"#F5A623", border:"none", borderRadius:8, padding:"10p
  kpiGrid: { display:"grid", gridTemplateColumns:"repeat(2,1fr)", gap:10, marg
  kpiCard: { background:"#111", border:"1px solid #1e1e1e", borderRadius:10, p
  kpiLabel: { color:"#666", fontSize:12, margin:"0 0 6px", textTransform:"upper

```

```

kpiValue: { fontSize:26, fontWeight:700, margin:"0 0 2px", letterSpacing:"-1p
kpiSub:    { color:"#555", fontSize:12, margin:0 },
budgetCard: { background:"#111", border:"1px solid #1e1e1e", borderRadius:10, p
budgetTrack:{ height:6, background:"#1e1e1e", borderRadius:99, overflow:"hidden
budgetFill: { height:"100%", background:"linear-gradient(90deg,#F5A623,#f97316)
sectionTitle:{ color:"#666", fontSize:11, letterSpacing:"1.2px", textTransform:
taskList:    { background:"#111", border:"1px solid #1e1e1e", borderRadius:10, o
taskRow:      { display:"flex", alignItems:"center", padding:"13px 14px", borderB
taskName:     { color:"#e5e5e5", fontSize:14, margin:"0 0 2px", fontWeight:600 },
taskMeta:     { color:"#555", fontSize:12, margin:0 },
badge:        { borderRadius:20, padding:"3px 10px", fontSize:11, fontWeight:600,
alertRow:     { display:"flex", alignItems:"flex-start", gap:10, background:"#1a0

// Chat
chatWrap:     { display:"flex", flexDirection:"column", height:"calc(100vh - 49p
messages:     { flex:1, overflowY:"auto", padding:"20px 16px 10px" },
msgRow:       { display:"flex", marginBottom:16, gap:10, alignItems:"flex-end" }
botAvatar:    { width:30, height:30, background:"#1a1a1a", border:"1px solid #2a
botBubble:    { background:"#141414", border:"1px solid #1e1e1e", borderRadius:"
userBubble:   { background:"#F5A623", borderRadius:"14px 14px 2px 14px", padding
typing:       { color:"#F5A623", letterSpacing:3, animation:"pulse 1s infinite"
suggestions:  { display:"flex", gap:6, padding:"8px 16px", overflowX:"auto", fle
suggBtn:      { background:"#141414", border:"1px solid #2a2a2a", borderRadius:2
inputRow:     { display:"flex", gap:8, padding:"10px 16px 16px", background:"#0a
chatInput:    { flex:1, background:"#141414", border:"1px solid #2a2a2a", border
sendBtn:      { background:"#F5A623", border:"none", borderRadius:10, width:44,
};

```